7.3 Контекстный логгер

Код: Структура и добавление полей

```go
package log

import (
    "context" "go.uber.org/zap"
    "go.uber.org/zap/zapcore"
)

type logCtx struct {
    OrderNr   string
    MessageID   string
    HandlerName  string
    MsgKey    string
}
type keyType int const key = keyType(0)


func Fields(ctx context.Context, fields []zapcore.Field) []zapcore.Field { l, ok :=
    ctx.Value(key).(logCtx)
    if !ok {
        return fields
    }
    if l.MessageID != "" {
        fields = append(fields, zap.String("message_id", l.MessageID))
    }
    if l.OrderNr != "" {
        fields = append(fields, zap.String("order_nr", l.OrderNr))
    }
    if l.HandlerName != "" {
        fields = append(fields, zap.String("handler_name", l.HandlerName))
    }
    if l.MsgKey != "" {
        fields = append(fields, zap.String("msg_key", l.MsgKey))
    }
    return fields
}
```

```
func msg(ctx context.Context, msg string) string { if l,
   ok := ctx.Value(key).(logCtx); ok {
      if l.HandlerName != "" {
         return l.HandlerName + ": " + msg
      }
   }
   return msg
}
```

Пояснение:

- `logCtx` — структура для хранения метаданных.
- `Fields` добавляет метаданные из контекста в поля лога Zap.
- `msg` модифицирует сообщение, добавляя `HandlerName` , если он задан.

Добавление метаданных в контекст

Функции `WithOrderNr` , `WithHandlerName` и `WithMsgKey` добавляют соответствующие метаданные в контекст, обновляя или создавая структуру `logCtx` .

```
package log

import (
   "context"
)

func WithOrderNr(ctx context.Context, orderNr string) context.Context {
   if l, ok := ctx.Value(key).(logCtx); ok {
      l.OrderNr = orderNr
      return context.WithValue(ctx, key, l)
   }
   return context.WithValue(ctx, key, logCtx{OrderNr: orderNr})
}

func WithHandlerName(ctx context.Context, handlerName string)
   context.Context { if l, ok := ctx.Value(key).(logCtx); ok {
      l.HandlerName = handlerName
      return context.WithValue(ctx, key, l)
   }
   return context.WithValue(ctx, key, logCtx{HandlerName: handlerName})
}
```

```
func WithMsgKey(ctx context.Context, msgKey string) context.Context {

   if l, ok := ctx.Value(key).(logCtx); ok { l.MsgKey = msgKey
      return context.WithValue(ctx, key, l)
   }
   return context.WithValue(ctx, key, logCtx{MsgKey: msgKey})
}
```

Пояснение:

- Каждая функция проверяет наличие `logCtx` в контексте.
- Если `logCtx` существует, она обновляет соответствующее поле; иначе создаёт новый контекст с `logCtx` .

Интерфейс `Logger` определяет методы для логирования на разных уровнях. Структура `LoggerCtx` реализует этот интерфейс, интегрируя контекстные метаданные с Zap.

Код: Интерфейс и реализация LoggerCtx

```
package log

import (
   "context"
   "go.uber.org/zap" "go.uber.org/zap/zapcore"
)

type Logger interface {
   Debug(ctx context.Context, msg string, fields ...zapcore.Field)
   Info(ctx context.Context, msg string, fields ...zapcore.Field) Warn(ctx
   context.Context, msg string, fields ...zapcore.Field) Error(ctx
   context.Context, msg string, fields ...zapcore.Field) Fatal(ctx
   context.Context, msg string, fields ...zapcore.Field)
}

type LoggerCtx struct {
log *zap.Logger
}
```

```
func New(log *zap.Logger) *LoggerCtx { return &LoggerCtx{
      log: log,

   }
}

func (l *LoggerCtx) Debug(ctx context.Context, msg string, fields
   ...zapcore.Field) { l.logCtx(ctx, zap.DebugLevel, msg, fields...)
}

func (l *LoggerCtx) Info(ctx context.Context, msg string, fields
   ...zapcore.Field) { l.logCtx(ctx, zap.InfoLevel, msg, fields...)
}

func (l *LoggerCtx) Warn(ctx context.Context, msg string, fields
   ...zapcore.Field) { l.logCtx(ctx, zap.WarnLevel, msg, fields...)
}

func (l *LoggerCtx) Error(ctx context.Context, msg string, fields
   ...zapcore.Field) { l.logCtx(ctx, zap.ErrorLevel, msg, fields...)
}

func ( imprenditori

l *LoggerCtx) Fatal(ctx context.Context, msg string, fields
   ...zapcore.Field) { l.logCtx(ctx, zap.FatalLevel, msg, fields...)
}

func (l *LoggerCtx) logCtx(ctx context.Context, lvl zapcore.Level, m string, f
   ...zapcore.Field) { f = Fields(ctx, f)
   l.log.Log(lvl, msg(ctx, m), f...)
}
```

Пояснение:

- Интерфейс `Logger` задаёт методы для каждого уровня логирования.
- `LoggerCtx` оборачивает `*zap.Logger` и использует `Fields` и `msg` для добавления метаданных.
- Метод `logCtx` объединяет контекстные поля и сообщение перед вызовом Zap.

Механизм `ErrorWithLogCtx` позволяет сохранять контекстные метаданные при оборачивании ошибок. Функции `WrapErrorCtx` и `ErrorCtx` обеспечивают интеграцию ошибок с контекстом.

Код: Обработка ошибок

```
package log

import (
    "context" "errors"
)

type ErrorWithLogCtx struct { next error
    ctx logCtx
}

func WrapErrorCtx(ctx context.Context, err error) error { lCtx, _ := ctx.Value(key).(logCtx)
    return &ErrorWithLogCtx{ next: err,
        ctx: lCtx,
    }

func ErrorCtx(ctx context.Context, err error) context.Context { var errorWithLCtx *ErrorWithLogCtx
    if errors.As(err, &errorWithLCtx) {
        ctx = context.WithValue(ctx, key, errorWithLCtx.ctx)
    }
    return ctx
}

func (e *ErrorWithLogCtx) Error() string { if e.next != nil {
        return e.next.Error()
    }
    return ""
}

func (e *ErrorWithLogCtx) Unwrap() error { if e.next != nil {
        return e.next
    }
    return nil
}
}
```

Пояснение:

- `WrapErrorCtx` оборачивает ошибку, сохраняя контекст `logCtx` .
- `ErrorCtx` извлекает контекст из ошибки типа `ErrorWithLogCtx` и добавляет его в новый контекст.
- Методы `Error` и `Unwrap` обеспечивают совместимость с интерфейсом `error` .

Код демонстрирует использование контекстного логгера в сервисе обработки заказов. Логгер передаётся в сервис, а контекст обогащается метаданными, такими как `OrderNr` и `MessageID`.

Код: Основной код проекта

```
package main

import (
   "context" "github.com/vostelmakh/ctxlog"
   "go.uber.org/zap"
   "service"
)

func main() {
   logger := zap.Must(zap.NewProduction())
   ctx := context.Background()
   ctxLogger := log.New(logger)
   orderNr := "ORD-12345"
   ctx = log.WithOrderNr(ctx, orderNr)
   ctx = log.WithMessageID(ctx, "MSG-1289")
   ctxLogger.Info(ctx, "Start handling request")
   orderService := service.NewOrderService(ctxLogger)
   err := orderService.UpdateOrder(ctx, orderNr)
   if err != nil {
      ctxLogger.Fatal(ctx, "Failed to update order", zap.Error(err))
   }
}
```

Код: Сервис обработки заказов

```
package service

import (
   "context" "github.com/vostelmakh/ctxlog"
)

type OrderService struct {
 logger log.Logger
}

func NewOrderService(logger log.Logger) *OrderService { return
   &OrderService{
      logger: logger,
   }
}

func (s *OrderService) UpdateOrder(ctx context.Context, orderNr string) error {
   s.logger.Info(ctx, "Start updating order")
   // Логика обновления заказа
   s.logger.Info(ctx, "Finish updating order")
   return nil
}
```

MERION

Вывод (примерный):

```
{"level":"info","ts":1622745180.123456,"msg":"Start handling
request","order_nr":"ORD-

12345","message_id":"MSG-1289"}

{"level":"info","ts":1622745180.123457,"msg":"Start updating
order","order_nr":"ORD-

12345","message_id":"MSG-1289"}
{"level":"info","ts":1622745180.123458,"msg":"Finish updating

order","order_nr":"ORD- 12345","message_id":"MSG-1289"}
```

Пояснение:

- Контекст обогащается метаданными (`OrderNr` , `MessageID`) с помощью `WithOrderNr` и `WithMessageID` .
- Логгер передаётся в `OrderService` и используется для логирования с учётом контекста.

Описание

Механизм обработки ошибок позволяет сохранять контекстные метаданные при возникновении ошибки и передавать их в логи.

```
package main

import (
    "context"
    "errors" "github.com/vostelmakh/ctxlog"
    "go.uber.org/zap"
)

func main() {
    logger := zap.Must(zap.NewProduction())
    ctx := context.Background()
    ctxLogger := log.New(logger)
    orderNr := "ORD-12345"
    ctx = log.WithOrderNr(ctx, orderNr)
    ctx = log.WithMessageID(ctx, "MSG-1289")
    ctx = log.WithHandlerName(ctx, "OrderHandler")
    ctxLogger.Info(ctx, "Start handling request")

    err := processOrder(ctx)
    if err != nil {
        wrappedErr := log.WrapErrorCtx(ctx, err)
        ctx = log.ErrorCtx(ctx, wrappedErr)
        ctxLogger.Error(ctx, "Failed to process order", zap.Error(wrappedErr))
    }
}

func processOrder(ctx context.Context) error {
 return errors.New("order processing failed")
}
```

Вывод (примерный):

```
{"level":"info","ts":1622745180.123456,"msg":"OrderHandler: Start handling

request","order_nr":"ORD-12345","message_id":"MSG-1289","handler_name":"OrderHan
dler"}
{"level":"error","ts":1622745180.123457,"msg":"OrderHandler: Failed to process
order","order_nr":"ORD-12345","message_id":"MSG-1289","handler_name":"OrderHandl
er","error":"order processing failed"}
```

Пояснение:

- Ошибка создаётся в `processOrder` и оборачивается с помощью `WrapErrorCtx`, сохраняя контекст.
- `ErrorCtx` восстанавливает контекст из ошибки для последующего логирования.
- Лог включает метаданные (`order_nr`, `message_id`, `handler_name`) и текст ошибки.

Основные выводы

- **Контекстный логгер**: Позволяет добавлять метаданные (`OrderNr, MessageID ,HandlerName , MsgKey `) через контекст, упрощая логирование в сложных системах.
- **Интеграция с Zap**: Использует производительность и JSON-формат Zap, добавляя контекстные поля автоматически.
- **Обработка ошибок**: сохраняет контекст при возникновении ошибок, что упрощает диагностику.
- **Гибкость**: Поддерживает все уровни логирования Zap и настраиваемые поля.

Рекомендации

- Используйте контекстный логгер в распределенных системах, где требуется отслеживание запросов по уникальным идентификаторам.
- Добавляйте метаданные в контекст на ранних этапах обработки запроса.
- Используйте `WrapErrorCtx` и `ErrorCtx` для сохранения контекста при обработке ошибок.
- Настраивайте Zap для вывода логов в файлы или системы мониторинга (например, ELK Stack).

Ключевые цитаты:

- Uber Zap Documentation: «Zap предоставляет быстрое, структурированное и уровневое логирование».
- SigNoz Guide: «Zap идеально подходит для структурированного логирования в современных приложениях».